

Kỹ Thuật Lập Trình Python

# Python Object-Oriented Programming



# CONTENT



## 1. Class

1. Định nghĩa class
2. Class object
3. Class và Instance variables
4. Instance object
5. Method object

## 2. Kế thừa (*Inheritance*)

## 3. Đa hình (*Polymorphism*)

## 4. Đóng gói (*Encapsulation*)

## 5. Trừu tượng (*Abstraction*)

Tham khảo từ: <https://docs.python.org/3/tutorial/classes.html>

# Tại sao vận dụng OOP?

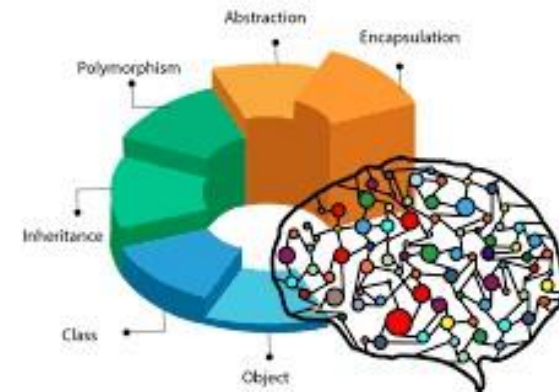
- Giải quyết các bài toán thực tế hiệu quả hơn.
- Có thể chia sẻ các lớp (class) nên mã nguồn tái sử dụng.
- Năng suất của chương trình tăng cao.
- Dữ liệu an toàn và bảo mật (trừu tượng).
- ...

# Giới thiệu OOP

## Object-Oriented Programming (OOP)

- Ý tưởng OOP?
- Là một phương pháp phân tích bài toán rất tiên tiến.
- Bài toán sẽ được phân tích thành các **đối tượng** mang đặc điểm và hành vi.
- OOP là phương pháp tư duy gần với thực tế cuộc sống của con người, xây dựng bài toán dựa vào mối quan hệ giữa các đối tượng tham gia vào bài toán.

OOPs (Object-Oriented Programming System)

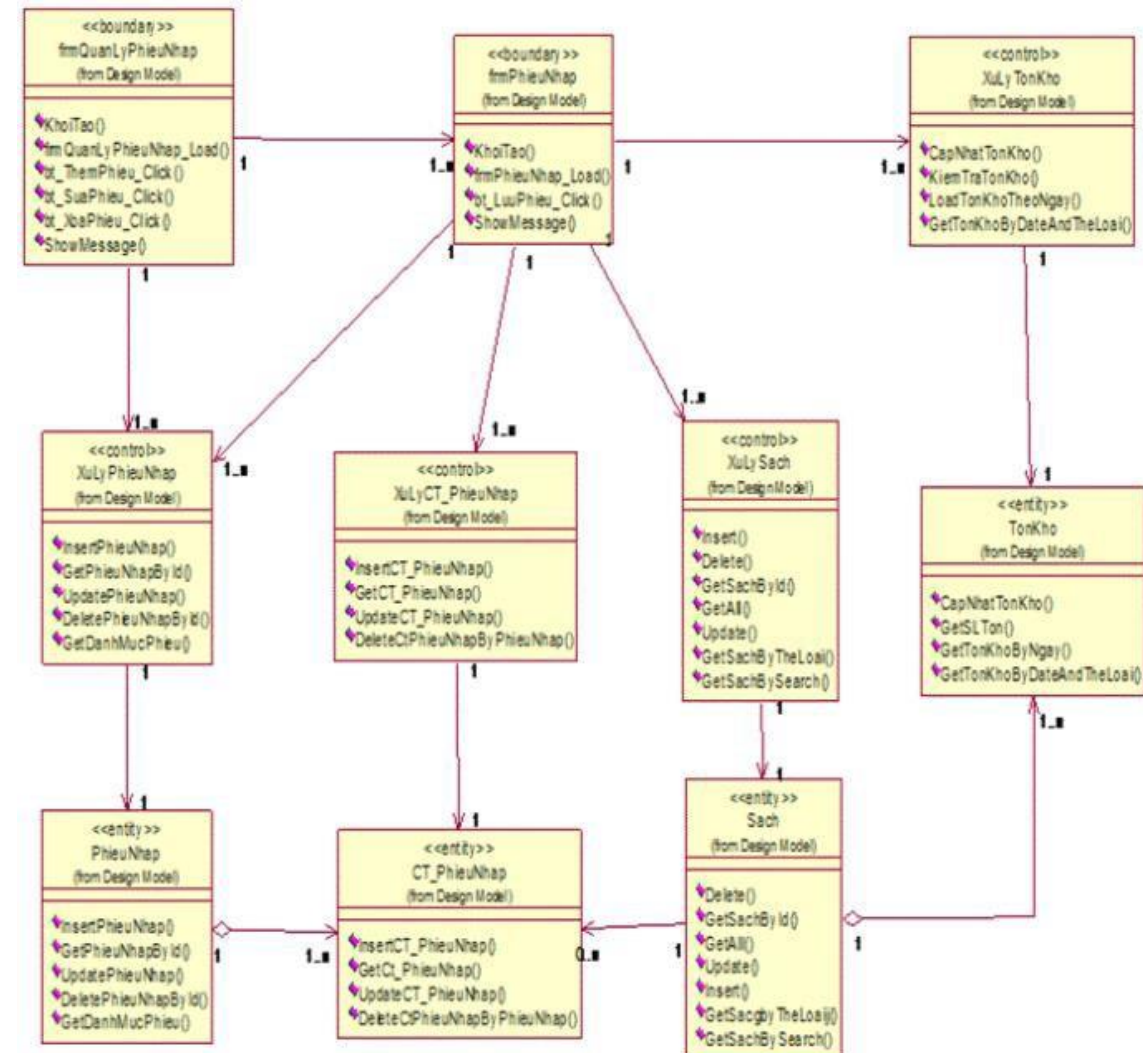
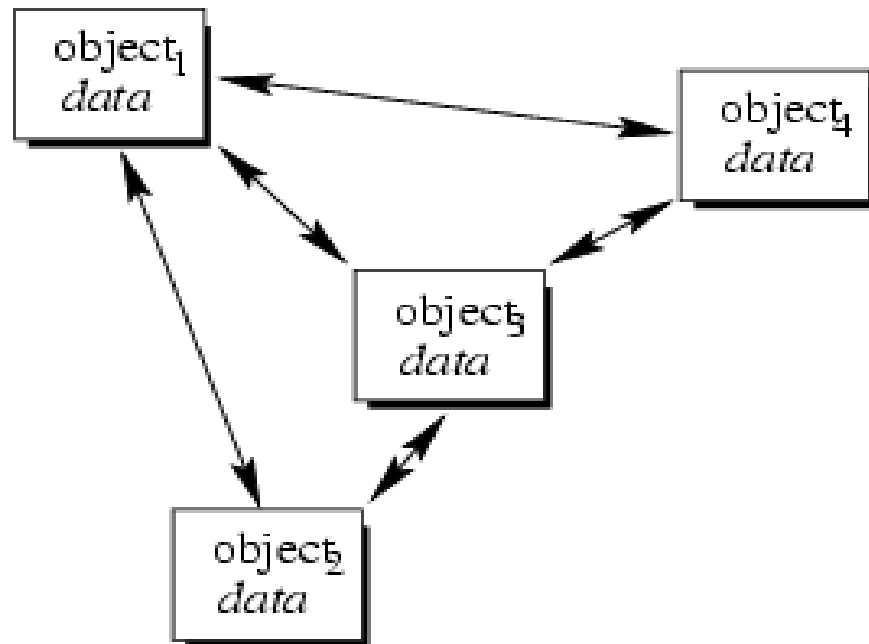


| Object                                                                               | Properties         | Methods     |
|--------------------------------------------------------------------------------------|--------------------|-------------|
|  | car.name = Fiat    | car.start() |
|                                                                                      | car.model = 500    | car.drive() |
|                                                                                      | car.weight = 850kg | car.brake() |
|                                                                                      | car.color = white  | car.stop()  |

# Giới thiệu OOP

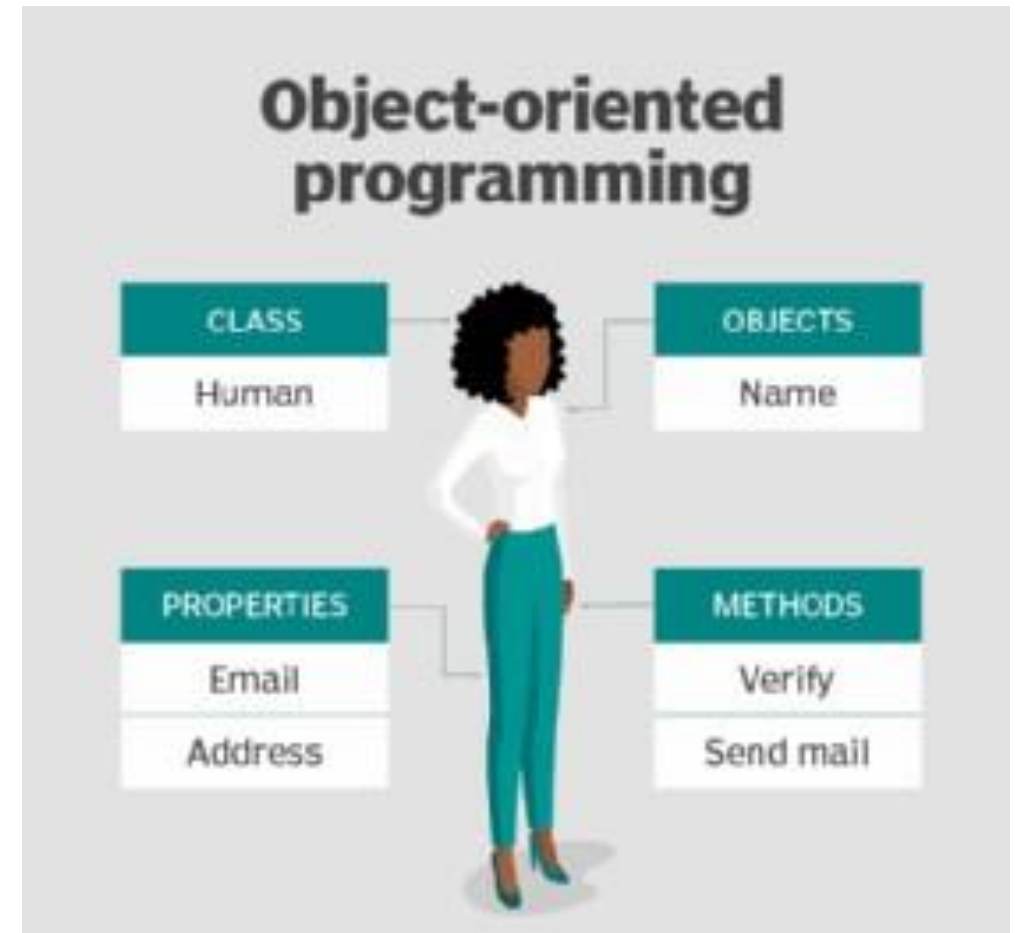
## Object-Oriented Programming (OOP)

- Lập trình hướng đối tượng là phương pháp lập trình dựa trên:
  - kiến trúc lớp (class)
  - và đối tượng (object)



# Các khái niệm

- Đối tượng - Objects
- Lớp - Class
- Thuộc tính - Properties
- Phương thức - Methods

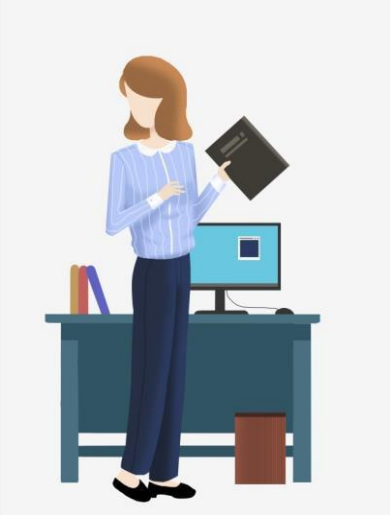


# Đối tượng

- Đối tượng (object) là một thực thể tham gia vào bài toán, có các **đặc điểm** và **hành vi**.
- Ví dụ: nhân viên, sinh viên, xe...

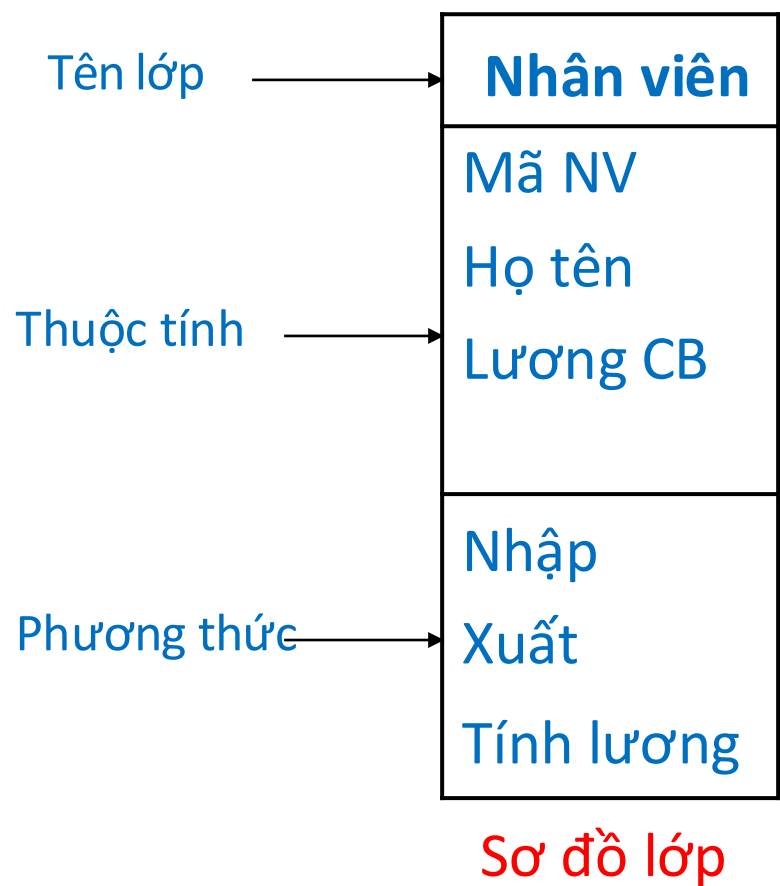
| Object                                                                              | Properties                      | Methods                  |
|-------------------------------------------------------------------------------------|---------------------------------|--------------------------|
|  | <code>car.name = Fiat</code>    | <code>car.start()</code> |
|                                                                                     | <code>car.model = 500</code>    | <code>car.drive()</code> |
|                                                                                     | <code>car.weight = 850kg</code> | <code>car.brake()</code> |
|                                                                                     | <code>car.color = white</code>  | <code>car.stop()</code>  |

## Ví dụ

| Object                                                                             | Properties                                                                                                   | Methods                                                                                                                              |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
|  | <code>maNV</code><br><code>hoTen</code><br><code>luongCB</code><br><code>heSo</code><br><code>luongHT</code> | <code>thongTin()</code><br><code>dangKy()</code><br><code>xetThuong()</code><br><code>dangKy()</code><br><code> tinhLuongHT()</code> |

# Sơ đồ đối tượng

- Sơ đồ đối tượng để mô tả các lớp đối tượng.
- Sơ đồ đối tượng bao gồm **sơ đồ lớp** và **sơ đồ thể hiện**.



**Đối tượng = TP. Dữ liệu + TP. Xử lý**

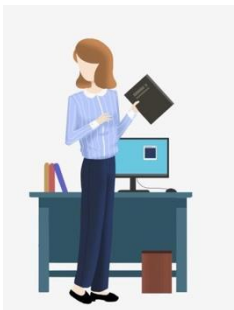
## ***Giới thiệu***

- Hỗ trợ biểu diễn cấu trúc của một hệ thống dựa trên các lớp.
- Trình bày mối quan hệ giữa các lớp.
- Biểu đồ mô tả lớp, thuộc tính, phương thức và mối quan hệ giữa các đối tượng trong hệ thống.
- Sơ đồ lớp giúp cho lập trình viên tổng quan về cấu trúc của hệ thống, hỗ trợ quá trình thiết kế, triển khai và bảo trì theo mô hình OOP.
- Có rất nhiều **công cụ hỗ trợ vẽ và tạo code**.
- Khóa học này dùng StarUML.

# Lớp - Class

## Giới thiệu

- Class là một bản thiết kế được dùng để **khởi tạo** đối tượng.
- Một class gồm có
  1. Thành phần dữ liệu (thuộc tính)
  2. Thành phần xử lý (các phương thức, hàm thành phần và khởi tạo...).
- Class thực chất là một **kiểu dữ liệu** do người dùng định nghĩa.



| NhanVien       |
|----------------|
| Mã NV          |
| Họ tên         |
| Lương cơ bản   |
| Xuất thông tin |

```
class NhanVien:

    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuat(self):
        print("\n")
        print("\t+Mã NV   :", self.maNV)
        print("\t+Họ tên  :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

Khởi tạo



## *Cú pháp*

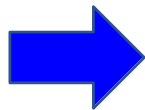
```
class ClassName:  
    <statement-1>  
    .  
    .  
    .  
    <statement-N>
```

*Khuyến nghị đặt tên theo quy ước PEP 8*

# Lớp - Class

## Cú pháp

```
class ClassName:  
    <statement-1>  
    .  
    .  
    .  
    <statement-N>
```



```
class ClassName:  
    <data attribute>  
    .  
    <constructor>  
    .  
    .  
    .  
    . . . . .  
    <method>  
    .  
    . . . . .
```

## Ví dụ

```
class NhanVien:
    soNV = 0

    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuat(self):
        print("\n")
        print("\t+Mã NV :", self.maNV)
        print("\t+Họ tên :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

```
class NhanVien:

    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuat(self):
        print("\n")
        print("\t+Mã NV :", self.maNV)
        print("\t+Họ tên :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

# Thuộc tính - Attribute

## ***Giới thiệu***

- Thuộc tính thể hiện **đặc điểm** của một đối tượng.
- Có thể hiểu là **một biến** bên trong class.
- Có một giá trị cụ thể và có miền giá trị đã được xác định.

## ***Ví dụ về một số thuộc tính***

- Hệ số lượng là một đặc điểm của một **nhân viên**, có miền giá trị xác định, là số thực. Ví dụ: 2.34, 2.69...
- Điểm là một một đặc điểm của một **sinh viên**, có miền giá trị xác định, là số thực. Ví dụ: 7.75, 8, 9.5...

# Thuộc tính - Attribute

## *Có hai loại thuộc tính*

- **Thuộc tính của lớp - Class Attribute**

- Là các biến được khai báo trực tiếp trong lớp, được chia sẻ bởi **tất cả các đối tượng** của lớp.
- Truy cập thuộc tính bằng **tên lớp** hoặc gọi từ **đối tượng**.

- **Thuộc tính của đối tượng - Instance Attribute**

- Là thuộc tính gắn liền với **một đối tượng** của lớp.
- Thuộc tính của đối tượng được định nghĩa trong hàm khởi tạo (constructor).
- Truy cập thuộc tính qua **đối tượng**.

# Thuộc tính - Attribute

## *Khai báo thuộc tính*

|                          |                                                           |
|--------------------------|-----------------------------------------------------------|
| Thuộc tính của lớp       | <code>tenThuocTinh = &lt;gán giá trị&gt;</code>           |
| Thuộc tính của đối tượng | <code>self.tenThuocTinh = &lt;giá trị khởi tạo&gt;</code> |

*Khuyến nghị đặt tên theo quy ước PEP 8*

# Thuộc tính - Attribute

*Ví dụ*

| NhanVien       |
|----------------|
| Mã NV          |
| Họ tên         |
| Lương cơ bản   |
| Xuất thông tin |

```
class NhanVien:  
    soNV = 0  
    def __init__(self):  
        NhanVien.soNV += 1
```

```
class NhanVien:  
    def __init__(self, maNV, hoTen, luongCB):  
        self.maNV = maNV  
        self.hoTen = hoTen  
        self.luongCB = luongCB
```

# Thuộc tính - Attribute

## Ví dụ

| NhanVien       |
|----------------|
| Mã NV          |
| Họ tên         |
| Lương cơ bản   |
| Xuất thông tin |

```
class NhanVien:
    soNV = 0

    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuat(self):
        print("\n")
        print("\t+Mã NV   :", self.maNV)
        print("\t+Họ tên  :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

# Phương thức khởi tạo - Constructor

## **Constructor là gì?**

- Constructor là một phương thức đặc biệt dùng để khởi tạo một đối tượng.
- Phương thức khởi tạo dùng để tạo giá trị cho thuộc tính.
- Không bắt buộc phải có phương thức khởi tạo trong class.

## **Phương pháp**

- Định nghĩa lại phương thức `__init__`:

```
def __init__(self, .....):  
    self.tenThuocTinh = <giá trị thuộc tính>  
    ....  
    ....
```

# Phương thức khởi tạo - Constructor

## Phương thức `__init__()`

- Dùng phương thức `__init__()` để khởi tạo một đối tượng. Ví dụ:

```
def __init__(self):  
    self.data = []
```

```
def __init__(self, *arg):  
    self.a = ...
```

```
def __init__(self, a, b):  
    self.a = a  
    self.b = b
```

```
def __init__(self, **kwargs):  
    self.a = ...  
    self.b = ...
```

```
def __init__(self, a=1, b=2):  
    self.a = a  
    self.b = b
```

```
def __init__(self, *arg, **kwargs):  
    self.a = ...  
    self.b = ...
```

## Phương thức `__init__()`

- Ví dụ tạo class biểu diễn phân số:

```
class PhanSo:  
    def __init__(self, tu, mau):  
        self.tu = tu  
        self.mau = mau
```

```
x = PhanSo(3, 5)  
x.tu, x.mau #Kết quả (3, 5)
```

## Phương thức `__init__()`

- Ví dụ:

```
class Complex:
    def __init__(self, realpart, imagpart):
        self.r = realpart
        self.i = imagpart
```

```
x = Complex(3.0, -4.5)
```

```
x.r, x.i (3.0, -4.5)
```

# Phương thức khởi tạo - Constructor

## Phương thức `__init__()`

- Ví dụ tạo class biểu diễn tọa độ:

```
class ToaDo:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y
```

```
p = ToaDo(3.67, 4.89)
p.x, p.y #Kết quả(3.67, 4.89)
```

# Phương thức khởi tạo - Constructor

## Ví dụ

| NhanVien       |
|----------------|
| Mã NV          |
| Họ tên         |
| Lương cơ bản   |
| Xuất thông tin |

```
class NhanVien:

    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuat(self):
        print("\n")
        print("\t+Mã NV   :", self.maNV)
        print("\t+Họ tên  :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

## ***Nạp chồng khởi tạo – Constructor overloading***

- Trong các ngôn ngữ khác, constructor overloading là một kỹ thuật cho phép một class có thể tạo ra các đối tượng bằng nhiều constructor khác nhau.
- Trong Python có hay không?
- Liên tưởng phương pháp tạo constructor bằng cách:
  - Tạo giá trị mặc định, kwargs
  - Kỹ thuật \*args
  - Kỹ thuật \*\*kwargs

# Phương thức khởi tạo - Constructor

## *Nạp chồng khởi tạo – Constructor overloading*

- Ví dụ

```
def __init__(self):  
    self.data = []
```

```
def __init__(self, *arg):  
    self.a = ...
```

```
def __init__(self, a, b):  
    self.a = a  
    self.b = b
```

```
def __init__(self, **kwargs):  
    self.a = ...  
    self.b = ...
```

```
def __init__(self, a=1, b=2):  
    self.a = a  
    self.b = b
```

```
def __init__(self, *arg, *kwargs):  
    self.a = ...  
    self.b = ...
```

# Phương thức khởi tạo - Constructor

## ***Bài tập***

- Dùng kỹ \*\*kwargs để xây dựng phương thức khởi tạo cho class NhanVien.
- Trong đó: maNV bắt buộc, còn lại tùy biến.

```
class NhanVien:  
  
    def __init__(self, maNV:int, **kwargs):  
        self.maNV = maNV  
        self.hoTen = kwargs['hoTen']  
        self.luongCB = kwargs['luongCB']
```

# Phương thức thành viên – Instance method

## ***Giới thiệu***

- Phương thức thể hiện **hành vi** của đối tượng (thể hiện hành động).
- Đối với mỗi class/đối tượng, ngoài các phương thức đã xây dựng để giải quyết vấn đề thì luôn có các phương thức cơ bản sau:
  - Nhập dữ liệu
  - Xuất, in ra dữ liệu
  - Truy vấn, cập nhật dữ liệu
- Nhập dữ liệu là phương thức nhập các giá trị đầu vào cho các thuộc tính
- Xuất, in ra dữ liệu là phương thức in ra các thông tin của các thuộc tính

## ***Ví dụ về phương thức:***

- Phương thức **tinhtDTB(self, ...)** thực hiện tính điểm trung bình cho các môn học.
- Phương thức **tinhtLuong(self, ...)** thực hiện tính lương nhân viên.

```
class NhanVien:
    def __init__(self, maNV: int, **kwargs):
        self.maNV = maNV
        self.hoTen = kwargs.get('hoTen', "Chưa cập nhật")
        self.luongCB = kwargs.get('luongCB', 0)

    def in_nv(self):
        print("Mã NV:", self.maNV)
        print("Họ tên:", self.hoTen)
        print("Lương CB:", self.luongCB)

nv1 = NhanVien(1, hoTen="Nguyễn Văn A", luongCB=5000000)
nv1.in_nv()

print()

nv2 = NhanVien(2)
nv2.in_nv()
```

✓ 0.0s

Python

Mã NV: 1  
Họ tên: Nguyễn Văn A  
Lương CB: 5000000

Mã NV: 2  
Họ tên: Chưa cập nhật  
Lương CB: 0

## *Cú pháp khai báo phương thức*

```
def tenPhuongThuc(self, <các khai báo input>):  
    """docstring: Mô tả hàm"""  
    #khởi lệnh  
    ...
```

*Khuyến nghị đặt tên theo quy ước PEP 8*

# Phương thức - Method

## Ví dụ

| NhanVien       |
|----------------|
| Mã NV          |
| Họ tên         |
| Lương cơ bản   |
| Xuất thông tin |

```
class NhanVien:

    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuat(self):
        print("\n")
        print("\t+Mã NV   :", self.maNV)
        print("\t+Họ tên   :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

## *Phương thức truy vấn*

- Phương thức truy vấn không làm thay đổi trạng thái hiện tại của đối tượng
- Sử dụng tiền tố “get” <tên của thuộc tính cần truy vấn>
- Phương thức truy vấn điều kiện nên dùng tiền tố “is”
- Ví dụ:

```
def getX(self):  
    return self.x
```

## *Phương thức cập nhật*

- Thay đổi trạng thái của đối tượng bằng cách sửa đổi một hoặc nhiều thành viên dữ liệu của đối tượng đó.
- Sử dụng tiền tố “set” <tên thuộc tính cần sửa>.
- Ví dụ:

```
def setX(self, x):  
    self.x = x
```

# Phương thức - Method

## Ví dụ

```
class ToaDo:
    def __init__(self, x, y) -> None:
        self.x = x
        self.y = y

    def getX(self) -> int:
        return self.x

    def setX(self, x) -> None:
        self.x = x

    def xuất(self) -> None:
        print("(" + self.x + ";", self.y + ")", sep="")
```

## ***Bài tập***

- Viết các phương thức get, set cho các thuộc tính lớp NhanVien

## Giới thiệu

- **Class method** là những phương thức đặc trưng cho class và gắn liền với **class** thay vì với **object**.
- Loại method này chủ yếu dùng để xử lý dữ liệu cho các thuộc tính trong class.
- Cách quy định thêm `@classmethod` trước khi viết phương thức. Và thay `self` thành `cls`
- Truy cập phương thức bằng **tên class** hoặc qua tên **đối tượng**.

## Ví dụ

```
class NhanVien:
    soNV = 0
    def __init__(self, maNV):
        self.maNV = maNV
        NhanVien.soNV += 1
```

```
@classmethod
def soLuong(cls):
    return cls.soNV
```

```
n1 = NhanVien('121')
n2 = NhanVien('122')
n3 = NhanVien('123')
```

```
print('Số nhân viên:', NhanVien.soLuong())  #Truy cập method class qua tên class
print('Số nhân viên:', n1.soLuong())        #Truy cập method class qua tên đối tượng
```

```
print('Số nhân viên:', NhanVien.soNV)      #Truy cập thuộc tính lớp qua tên class
```

## Static method

- **Static method** là những phương thức **tự do** đặt bên trong class, **không ràng buộc** về dữ liệu so với **class** hay **object**.
- Tạo static method trong class bằng cách dùng khai báo `@staticmethod` trước khi viết phương thức.
- Truy cập phương thức bằng **tên class**.

# Static method

## Ví dụ

```
class NhanVien:
    soNV = 0
    def __init__(self, maNV):
        self.maNV = maNV
        NhanVien.soNV += 1

    @classmethod
    def soLuong(cls):
        return cls.soNV

    @staticmethod
    def tienThuong(tien):
        return tien * 1.2

print('Tiền thưởng', NhanVien.tienThuong(8_000_000))
```

# Khởi tạo đối tượng

## Cú pháp

<tên đối tượng> = **ClassName**(...)

| NhanVien       |
|----------------|
| Mã NV          |
| Họ tên         |
| Lương cơ bản   |
| Xuất thông tin |



```
class NhanVien:

    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuat(self):
        print("\n")
        print("\t+Mã NV   :", self.maNV)
        print("\t+Họ tên   :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

constructor



## Ví dụ

```
nv1 = NhanVien(122, 'Nguyễn Văn A', 8_600_000)
nv2 = NhanVien(123, 'Nguyễn Văn B', 6_500_000)
nv3 = NhanVien(124, 'Nguyễn Văn C', 9_800_000)
```

## ***Lớp - Class***

- Thuộc tính
  - Thuộc tính của đối tượng – Instance Attribute
  - Thuộc tính của lớp – Class Attribute
- Phương thức
  - Phương thức khởi tạo – Constructor
  - Phương thức của đối tượng – Instance method – Method
  - Phương thức của lớp – Class method
  - Phương thức tĩnh – Static method

# BÀI TẬP ÁP DỤNG

1. Giải phương trình bậc nhất  $aX + b = 0$
2. Giải phương trình bậc 2  $aX^2 + bX + c = 0$
3. Giải phương trình trùng phương  $aX^4 + bX^2 + c = 0$

**Code it!**

Chú ý:

- Tất cả các bài được thiết kế theo phương pháp hướng đối tượng.

## Bài tập QLNV: Áp dụng OOP (class - obj)

Viết chương trình quản lý nhân viên cho một công ty có tối đa 300 nhân viên. Công ty gồm 2 loại nhân viên: nhân viên văn phòng và nhân viên bán hàng. Công ty cần quản lý các thông tin sau của nhân viên: mã nhân viên, họ và tên, lương cơ bản, lương hằng tháng.

Lương hằng tháng tính bằng công thức sau:

- NV văn phòng = lương cơ bản + **số ngày là việc** \* 150\_000đ
- NV bán hàng = lương cơ bản + **số sản phẩm** \* 18\_000đ

Chương trình đáp ứng các yêu cầu sau:

1. Khởi tạo tự động các nhân viên. Chỉ cung cấp: maNV, hoTen, luongCB.
2. Tính lương hằng tháng.
3. Xuất các nhân viên (mã nhân viên, họ tên, lương cơ bản, **lương hằng tháng**)
4. Tìm nhân viên theo mã nhân viên
5. Tìm nhân viên có lương hằng tháng cao nhất
6. Tìm nhân viên bán hàng có lương hằng tháng thấp nhất

# Hướng dẫn

- Phạm vi kiến thức: Dùng OOP (class - obj)
- Thiết kế: Mọi yêu cầu đều áp dụng OOP.
- Bao nhiêu class? => 3
- Liệt kê tên class? => NVVanPhong, NVBanHang, CongTy

| NVVanPhong                                             |
|--------------------------------------------------------|
| Mã NV<br>Họ tên<br>Lương cơ bản<br>Số ngày<br>Lương HT |
| Xuất thông tin<br>Tính lương                           |

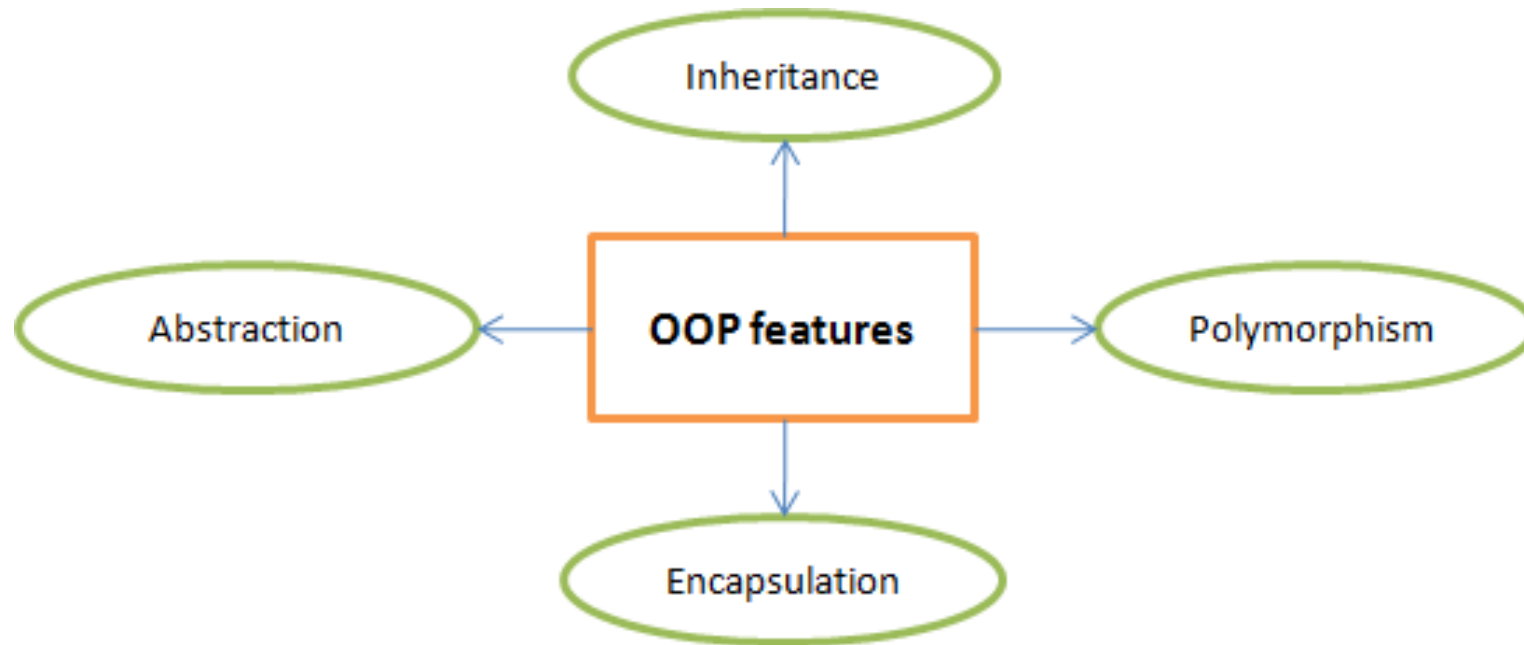
| NVBanHang                                                  |
|------------------------------------------------------------|
| Mã NV<br>Họ tên<br>Lương cơ bản<br>Số sản phẩm<br>Lương HT |
| Xuất thông tin<br>Tính lương                               |

| CongTy                                                                 |
|------------------------------------------------------------------------|
| Mã công ty<br>Ten ct<br>Danh sách nhân viên                            |
| 1. Load/khởi tạo NV<br>2. Xuất các nv<br>3. Tính lương<br>4. Tìm<br>5. |

# Đặc tính của OOP

## *Giới thiệu*

- Các tính chất quan trọng ==> điểm mạnh của OOP



## ***Giới thiệu***

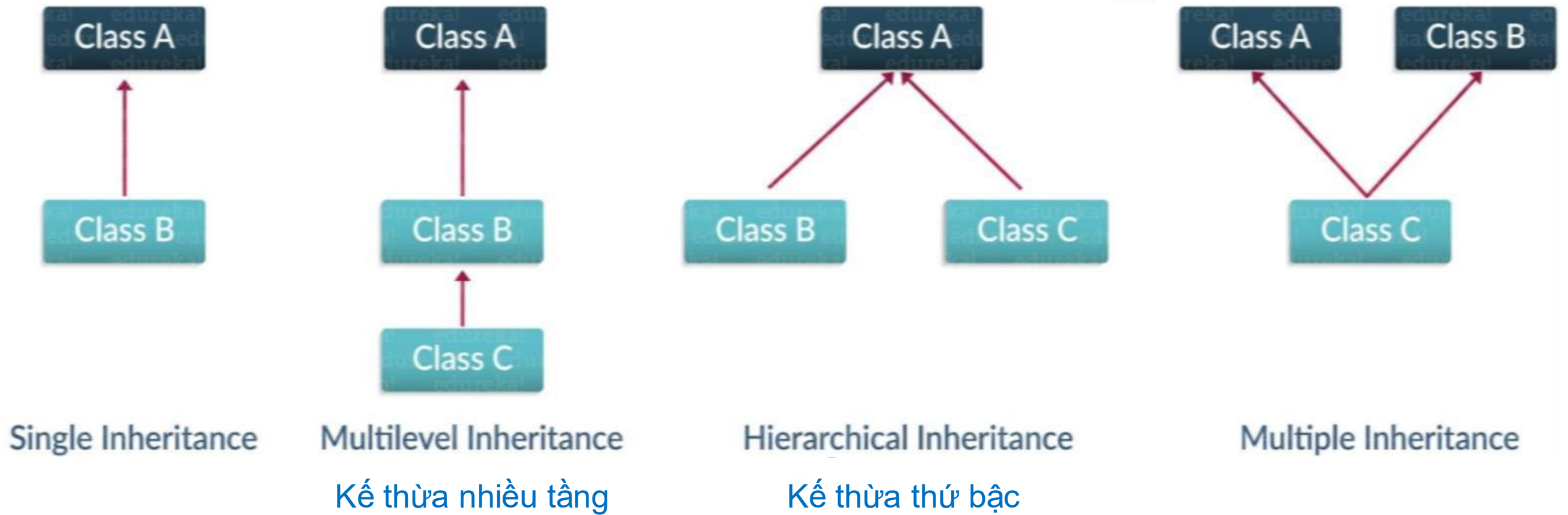
- Kế thừa là sự thừa hưởng lại các thuộc tính, phương thức mà không cần khai báo lại các thuộc tính, phương thức.
- Kế thừa giúp tái sử dụng lại mã nguồn giúp lập trình viên dễ dàng rà soát bảo trì mã nguồn.

```
class A:
    def __init__(self, a):
        self.a = a

class B(A):
    def __init__(self, a, b):
        super().__init__(a)
        self.b = b

    def xuất(self):
        print(self.a, self.b)
```

## Các kiểu kế thừa



# Tính kế thừa

## Minh họa



Single Inheritance

```
class A:
    def __init__(self, a):
        self.a = a

class B(A):
    def __init__(self, a, b):
        super().__init__(a)
        self.b = b

    def xuất(self):
        print(self.a, self.b)
```

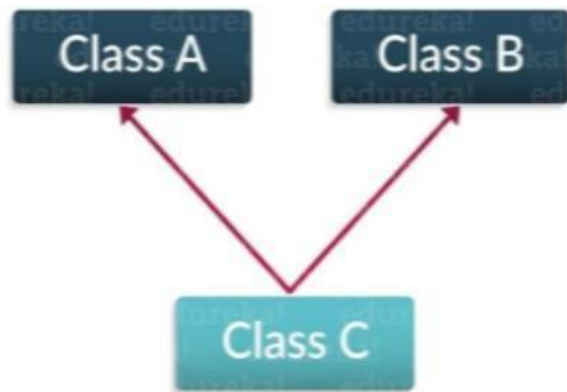
```
class A:
    def __init__(self, a):
        self.a = a

class B(A):
    def __init__(self, a, b):
        A.__init__(self, a)
        self.b = b

    def xuất(self):
        print(self.a, self.b)
```

# Tính kế thừa

## Minh họa



Multiple Inheritance

```
class A:
    def __init__(self, a):
        self.a = a

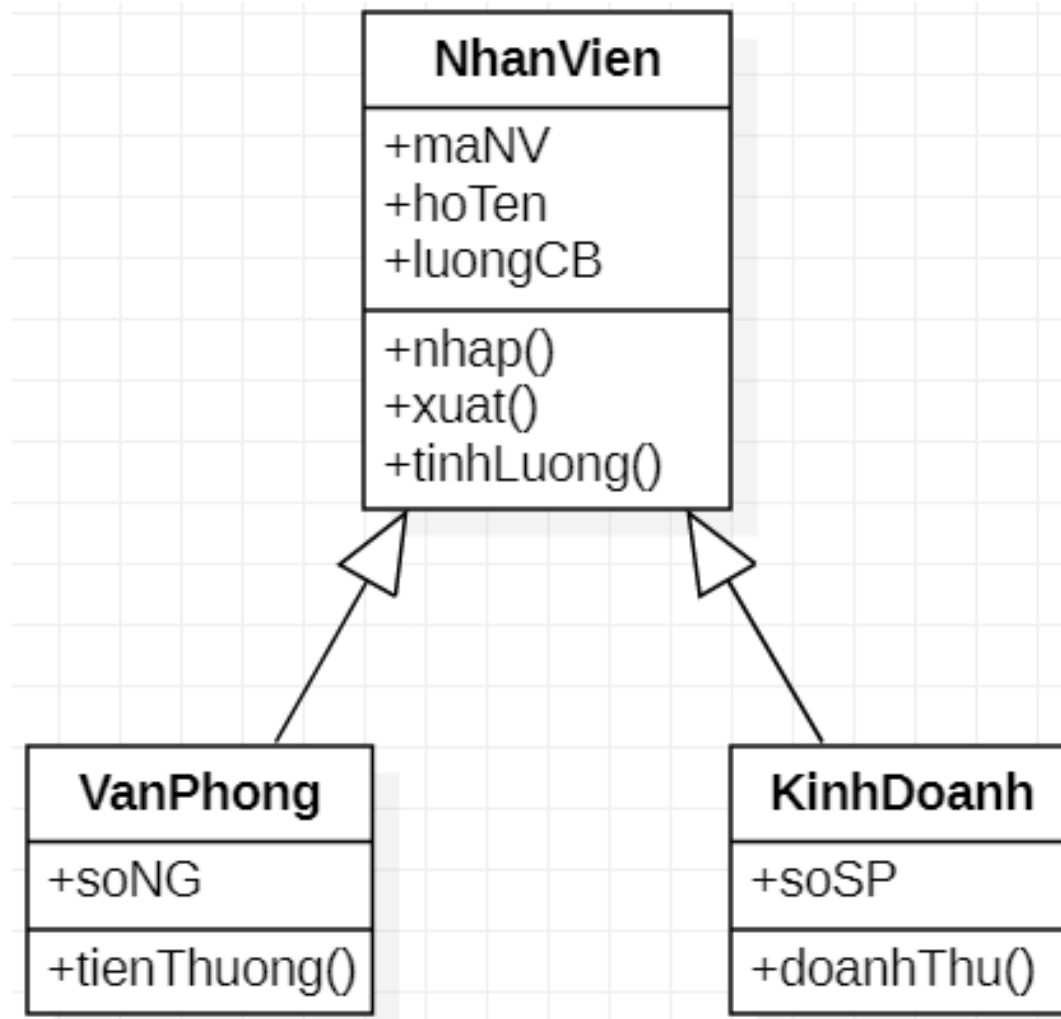
class B:
    def __init__(self, b):
        self.b = b

class C(A, B):
    def __init__(self, a, b, c):
        A.__init__(self, a)
        B.__init__(self, b)
        self.c = c

    def xuất(self):
        print(self.a, self.b, self.c, end="\t")
```

# Tính kế thừa

## Sơ đồ lớp



# Tính kế thừa

## Cài đặt

```
class NhanVien:
    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self.hoTen = hoTen
        self.luongCB = luongCB

    def xuất(self):
        print("\n")
        print("\t+Mã NV   :", self.maNV)
        print("\t+Họ tên   :", self.hoTen)
        print("\t+Lương CB:", self.luongCB)
```

```
class VanPhong(NhanVien):
    def __init__(self, maNV, hoTen, luongCB, soNG):
        super().__init__(maNV, hoTen, luongCB)
        self.soNG = soNG

class KinhDoanh(NhanVien):
    def __init__(self, maNV, hoTen, luongCB, soSP):
        super().__init__(maNV, hoTen, luongCB)
        self.soSP = soSP

vp1 = VanPhong(121, 'Nguyễn Văn A', 6_500_000, 25)
kd1 = KinhDoanh(122, 'Nguyễn Văn B', 9_500_000, 68)

vp1.xuất()
kd1.xuất()
```

## ***Nạp chồng phương thức***

- Trong các ngôn ngữ khác, nạp chồng là các phương thức có cùng tên, nhưng khác nhau về số lượng tham số/đối số truyền vào.
- Kỹ thuật nạp chồng làm tăng sự linh hoạt cho các phương thức.
- Python không hỗ trợ nạp chồng phương thức theo tham số.
- Xét ví dụ sau:

```
class A:  
    def a(self, b):  
        return b  
    def a(self, b, c):  
        return b + c  
  
a = A()  
print(a.a(1, 2))  
  
print(a.a(1)) #Báo lỗi
```

## ***Để phương thức linh hoạt sử dụng***

- Kỹ thuật tạo giá trị mặc nhiên

```
class A:  
    def a(self, b=0, c=0):  
        return b + c
```

```
a = A()
```

```
print(a.a(1, 2))
```

```
print(a.a(1))
```

```
print(a.a())
```

## *Để phương thức linh hoạt sử dụng*

- Kỹ thuật \*args

```
class A:  
    def a(self, *arg):  
        return sum(arg)
```

```
a = A()
```

```
print(a.a(1, 2, 3))
```

```
print(a.a(1, 2))
```

```
print(a.a(1))
```

```
print(a.a())
```

## *Để phương thức linh hoạt sử dụng*

- Kỹ thuật `**kwargs`

```
class A:  
    def a(self, **kwargs):  
        return kwargs  
  
obj = A()  
  
print(obj.a(x=1))  
print(obj.a(x=1, y=2))  
print(obj.a(x=1, y=2, z=3))
```

## *Ghi đè phương thức trong class con*

- **Ghi đè hoàn toàn:**

- Là việc xóa bỏ và xây dựng lại toàn bộ nội dung của phương thức trong class cha.
- Khi nào phải ghi đè hoàn toàn?

- **Ghi đè không hoàn toàn:**

- Là kế thừa lại một phần phương thức trong class cha và xây dựng thêm nội dung cho phương thức con.
- Sử dụng `super().tenPhuongThuc(...)`, `TenClassCha.tenPhuongThuc(self, ...)` để gọi đến phương thức trong class cha

## ***Ghi đè phương thức trong class con***

1. Một phương thức a() được xây dựng trong class cha (class A).
2. Tiếp theo, class B kế thừa class A.
3. Và trong class B, xây dựng thêm phương thức a() cùng tên.

=> Phương thức a() trong B sẽ ghi đè phương thức a() trong A

```
class A:
    def a(self):
        print("Phương thức a trong lớp A")

class B(A):
    def a(self):
        print("Phương thức a trong lớp B")

b = B()
b.a()
```

## *Ghi đè phương thức trong class con*

```
class A:  
    def a(self):  
        print("Phương thức a trong lớp A")  
  
class B(A):  
    def a(self):  
        super().a()  
        print("Phương thức a trong lớp B")  
  
b = B()  
b.a()
```

## Minh họa ghi đè hoàn toàn

```
class A:
    def a(self, a=1, b=2):
        print('Class A:')
        print('\ta = ', a)
        print('\tb = ', b)

class B(A):
    def a(self):
        print('Class B')
```

```
a = A()

a.a(3, 4)
a.a(1, 2)
a.a(1)
a.a()

b = B()
b.a()
```

## Minh họa ghi đè không hoàn toàn

```
class A:
    def a(self, a=1, b=2):
        print('Class A:')
        print('\ta = ', a)
        print('\tb = ', b)

class B(A):
    def a(self):
        print('Class B')
        super().a()
```

```
a = A()
a.a(3, 4)
a.a(1, 2)
a.a(1)
a.a()
```

```
b = B()
b.a()
```

# Tính đóng gói

## ***Khái niệm***

- Tính đóng gói (encapsulation) trong OOP để ẩn thông tin và hành vi của đối tượng bên trong lớp.
- Để bảo vệ thuộc tính và các phương thức của lớp, ngăn không cho truy cập trực tiếp và sửa đổi.

## ***Lợi ích của tính đóng gói***

- Bảo vệ dữ liệu.
- Ứng dụng nguyên tắc trừu tượng.
- Quản lý sự thay đổi.
- Cung cấp giao diện rõ ràng.
- Tăng tính bảo mật và kiểm soát truy cập.

# Phạm vi truy cập - Access modifiers

## *Phạm vi truy cập?*

- Là việc quy định phạm vi, quyền hạn của thuộc tính và phương thức.

## *Ví dụ*

- Một đối tượng A được qui định phạm vi về thuộc tính và phương thức
- Thì các đối tượng khác sẽ có mức độ truy cập vào đối tượng A là khác nhau.

# Phạm vi truy cập - Access modifiers

- Khi khai báo/xây dựng một class có thể dùng các **ký pháp** để thể hiện **phạm vi truy cập** của thuộc tính, phương thức...
- Phạm vi truy cập:
  - Public: để trống
  - Protected: ký pháp `_`
  - Private: ký pháp `__`
- Phạm vi truy cập sử dụng để thể hiện mức độ riêng tư trong khai báo thuộc tính, constructor, phương thức...

```
class NhanVien:
    def __init__(self, maNV, hoTen, luongCB):
        self.maNV = maNV
        self._hoTen = hoTen
        self.__luongCB = luongCB

    def xuất(self):
        print("\n")
        print("\t+Mã NV   :", self.maNV)
        print("\t+Họ tên  :", self._hoTen)
        print("\t+Lương CB:", self.__luongCB)
```

# Phạm vi truy cập - Access modifiers

## Mô tả

| Quyền hạn - Phạm vi | Mô tả                                                                                                                                                                              |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| protected           | <ul style="list-style-type: none"><li>Truy cập từ bên trong lớp và các lớp con kế thừa của lớp đó.</li></ul>                                                                       |
| private             | <ul style="list-style-type: none"><li>Không thể truy cập bên ngoài lớp.</li><li>Không thể truy cập từ các lớp con kế thừa của lớp đó.</li><li>Truy cập từ bên trong lớp.</li></ul> |
| public              | <ul style="list-style-type: none"><li>Truy cập mọi phạm vi.</li></ul>                                                                                                              |

# Phạm vi truy cập - Access modifiers

## ***Cú pháp khai báo***

- Khai báo quyền hạn cho thuộc tính, phương thức.

| Quyền hạn – Phạm vi | Phương pháp | Ví dụ                                                               |
|---------------------|-------------|---------------------------------------------------------------------|
| protected           | —           | <code>self._maNV</code><br><code>def _tinhLuong(self, ...)</code>   |
| private             | --          | <code>self.__maNV</code><br><code>def __tinhLuong(self, ...)</code> |
| public              | Để trống    | <code>self.maNV</code><br><code>def tinhLuong(self, ...)</code>     |

# Phạm vi truy cập - Access modifiers

## *Bảng tóm tắt quyền hạn và phạm vi*

|                  | Internal class                      | Kế thừa                             | Trong cùng package                  | Khác package                        |
|------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|
| <b>private</b>   | <input checked="" type="checkbox"/> |                                     |                                     |                                     |
| <b>protected</b> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| <b>public</b>    | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |

## *Ví dụ minh họa*

- Xem demo GV

# Khuyến nghị

## *Khuyến nghị khi khai báo quyền hạn cho thuộc tính, phương thức*

- **Đối với thuộc tính** (*nên khai báo 2 quyền*)
  - **private**: nếu thuộc tính chỉ dùng riêng trong một class/đối tượng
  - **protected**: nếu thuộc tính đó được các đối tượng con kế thừa
- **Đối với phương thức** (*tùy thuộc vào phạm vi mà có thể khai báo quyền*)
  - **private**: nếu các phương thức chỉ sử dụng trong nội bộ.
  - **protected**: nếu các phương thức được sử dụng trong đối tượng con.
  - **public**: nếu các phương thức được sử dụng ở mọi nơi, kể cả việc khác package.

# Phạm vi truy cập - Access modifiers

***Tại sao nên khai báo quyền hạn thuộc tính là private hoặc protected?***

- **Bảo vệ toàn vẹn dữ liệu** của một đối tượng, ngăn không cho đối tượng khác có thể truy cập để thay đổi dữ liệu.
- Nếu đối tượng B truy cập đến thuộc tính của đối tượng A sẽ có 2 trường hợp:
  - Trường hợp 1: Lấy thuộc tính của đối tượng A
  - Trường hợp 2: Sửa thuộc tính của đối tượng A

# Lấy thuộc tính

## *Trường hợp 1: Lấy thuộc tính của đối tượng*

- Trong class A khai báo quyền hạn thuộc tính là private thì tại class B sẽ không truy cập được thuộc tính.
- Trong trường hợp này, muốn lấy thông tin thuộc tính trong A thì phải **xây dựng thêm phương thức truy vấn `getTen...(self)`** để lấy giá trị thuộc tính.
- Tham khảo cách sau:

```
def getTenThuocTinh(self):  
    return self.__TenThuocTinh  
  
@property  
def name(self):  
    return self.__TenThuocTinh
```

## Ví dụ

```
class A:  
    def __init__(self, value):  
        self.__value = value  
  
    def getValue(self):  
        return self.__value  
  
    @property  
    def value(self):  
        return self.__value
```

```
a = A(39)  
print(a.value)  
print(a.getValue())
```

# Lấy thuộc tính

## Ví dụ

```
class NhanVien:
    def __init__(self, name):
        self.__name = name

    def getName(self):
        return self.__name

    @property
    def name(self):
        return self.__name
```

```
n1 = NhanVien('Văn A')
n2 = NhanVien('Văn B')

print('Tên nhân viên n2:', n2.name)
print('Tên nhân viên n1:', n1.getName())
```

# Cập nhật thuộc tính

## *Trường hợp 2: Sửa thuộc tính của đối tượng*

- Trong class C khai báo quyền hạn thuộc tính là **private** thì tại class B sẽ không truy cập được thuộc tính.
- Trong trường hợp này, muốn **cập nhật** giá trị thuộc tính trong C thì phải xây dựng thêm phương thức **setTen...()** để **cập nhật** giá trị thuộc tính.
- Tham khảo cách sau:

```
def setValue(self, value):  
    self.__value = value
```

```
@property  
def value(self):  
    return self.__value
```

```
@property  
def value(self):  
    return self.__value
```

- Chú ý: việc thiết kế phương thức set không được tùy tiện.

# Cập nhật thuộc tính

## Ví dụ

```
class C:
    def __init__(self, value):
        self.__value = value

    def setValue(self, value):
        self.__value = value

    @property
    def value(self):
        return self.__value

    @value.setter
    def value(self, valueNew):
        self.__value = valueNew
```

```
a = C(39)
print(a.value)
```

```
a.value = 79
print(a.value)
```

```
a.setValue(99)
print(a.value)
```

# Cập nhật thuộc tính

## Ví dụ

```
class NhanVien:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, nameNew):
        self.__name = nameNew

    def setName(self, nameNew):
        self.__name = nameNew
```

```
n1 = NhanVien('Văn A')
n2 = NhanVien('Văn B')
print('Tên nhân viên n1:', n1.name)
print('Tên nhân viên n2:', n2.name)

n1.name = 'Văn C'
print('Tên nhân viên n1:', n1.name)

n2.setName('Văn D')
print('Tên nhân viên n2:', n2.name)
```

## Decorator?

- Là một kỹ thuật thiết kế mẫu giúp đối tượng trở nên linh hoạt.

## Built-in Decorators

- Python cung cấp các decorator có sẵn để hỗ trợ định nghĩa property, class method, static method... cho class.

| Decorator     | Mô tả                                                  |
|---------------|--------------------------------------------------------|
| @property     | Định nghĩa phương thức để get và set các property.     |
| @classmethod  | Định nghĩa phương thức class method gọi bằng tên class |
| @staticmethod | Định nghĩa các static method.                          |

# @property

```
class NhanVien:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, value):
        self.__name = value

    @name.deleter # property-name.deleter decorator
    def name(self, value):
        print('Đang xóa...', self.__name)
        del self.__name
        print('Đã xóa.')
```

```
n1 = NhanVien('Văn A')
n2 = NhanVien('Văn B')

print('Tên nhân viên n2:', n2.name)
n1.name = 'Văn C'
print('Tên nhân viên n1:', n1.name)
```

# Bài tập QLNV: Áp dụng OOP (Tính đóng gói, kế thừa)

Viết chương trình quản lý nhân viên cho một công ty có tối đa 300 nhân viên. Công ty gồm 2 loại nhân viên: nhân viên văn phòng và nhân viên bán hàng. Công ty cần quản lý các thông tin sau của nhân viên: mã nhân viên, họ và tên, lương cơ bản, lương hằng tháng.

Lương hằng tháng tính bằng công thức sau:

- NV văn phòng = lương cơ bản + **số ngày là việc** \* 150\_000đ
- NV bán hàng = lương cơ bản + **số sản phẩm** \* 18\_000đ

Chương trình đáp ứng các yêu cầu sau:

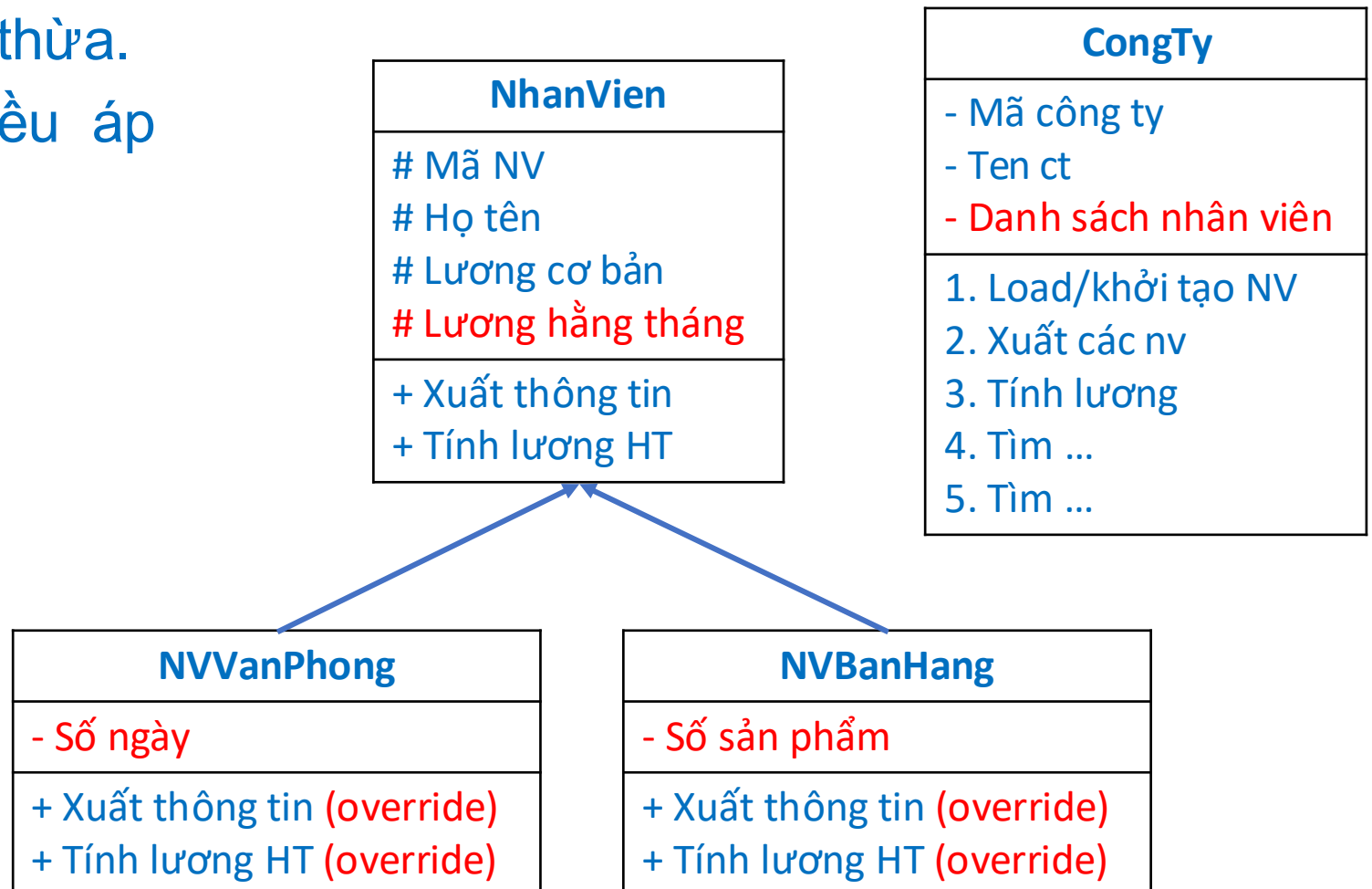
1. Khởi tạo tự động các nhân viên. Chỉ cung cấp: maNV, hoTen, luongCB.
2. Xuất các nhân viên (mã nhân viên, họ tên, lương cơ bản, **lương hằng tháng**).
3. Tính lương hằng tháng các nhân viên.
4. Tìm nhân viên theo mã nhân viên.
5. Cập nhật lương cơ bản theo mã nhân viên.
6. Cập nhật số ngày làm của nhân viên văn phòng theo mã nhân viên.
7. Tìm nhân viên có lương hằng tháng cao nhất.
8. Tìm nhân viên bán hàng có lương hằng tháng thấp nhất.

# Hướng dẫn

- Phạm vi kiến thức: Dùng OOP áp dụng tính đóng gói, kế thừa.
- Thiết kế: Mọi yêu cầu đều áp dụng OOP.
- Bao nhiêu class? => 4
- Liệt kê tên class? =>
  - NhanVien
  - NVVanPhong
  - NVBanHang
  - CongTy

Chú ý khi vẽ sơ đồ class

- private
- # protected
- + public



## ***Giới thiệu***

- Đa hình trong OOP cho phép các đối tượng của các lớp khác nhau => có các phương thức cùng tên với nhau nhưng thực hiện xử lý khác nhau.
- Đa hình trong Python thường được thực hiện thông qua việc sử dụng kế thừa, ghi đè phương thức và sử dụng các phương thức có sẵn.
- Vậy vậy, tính đa hình được thể hiện thông qua **tính kế thừa** và ghi đè phương thức.

## ***Ý nghĩa của tính đa hình***

- Kiểm soát các phương thức dùng chung của các lớp giống hoặc khác nhau về **hành vi bản chất**.

## Đa hình trong kế thừa

- Các lớp con có thể kế thừa các phương thức từ lớp cha và có thể ghi đè bằng cách định nghĩa lại phương thức của lớp con.
- Khi tạo ra một đối tượng rồi gọi phương thức từ đối tượng vừa tạo:
  - Phương thức của lớp con sẽ được gọi nếu đã được định nghĩa trong lớp con.
  - Ngược lại phương thức của lớp cha sẽ được gọi.

```
if __name__ == '__main__':  
    b = B(1, 2)  
    b.methodA()  
  
    c = C(3, 4)  
    c.methodA()
```

```
class A:  
    def __init__(self, a):  
        self._a = a  
  
    def methodA(self):  
        print('Class A. a = ', self._a)
```

```
class B(A):  
    def __init__(self, a, b):  
        super().__init__(a)  
        self.__b = b  
  
    def methodA(self):  
        print('Class B. a = ', self._a, 'b = ', self.__b)
```

```
class C(A):  
    def __init__(self, a, b):  
        super().__init__(a)  
        self.__b = b
```

## Ví dụ

| Class       | Phương thức                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Hinh        | <pre>def chuVi(self):<br/>    pass<br/><br/>def dienTich(self):<br/>    pass</pre>                                                     |
| HinhVuong   | <pre>def chuVi(self):<br/>    return self.canh * 4<br/><br/>def dienTich(self):<br/>    return self.canh ** 2</pre>                    |
| HinhChuNhat | <pre>def chuVi(self):<br/>    return (self.dai + self.rong) * 2<br/><br/>def dienTich(self):<br/>    return self.dai * self.rong</pre> |

## Nhận xét

- Cùng là một phương thức tên **chuVi** nhưng sẽ thực thi khác nhau cho từng loại đối tượng.

# Tính đa hình

## Ví dụ

```
class Hình:  
    def chuVi(self):  
        return None  
    def dienTich(self):  
        return None
```

```
class Vuong(Hình):  
    def __init__(self, canh) -> None:  
        self.canh = canh  
  
    def chuVi(self):  
        return self.canh * 4  
  
    def dienTich(self):  
        return self.canh ** 2
```

```
class ChuNhat(Hình):  
    def __init__(self, dai, rong) -> None:  
        self.dai = dai  
        self.rong = rong  
  
    def chuVi(self):  
        return (self.dai + self.rong) * 2  
  
    def dienTich(self):  
        return self.dai * self.rong
```

## Ví dụ

```
v = Vương(3)
print(v.chuVi())
```

```
cn = ChuNhat(3, 4)
print(cn.dienTich())
```

```
def chuVi(obj):
    return obj.chuVi()
```

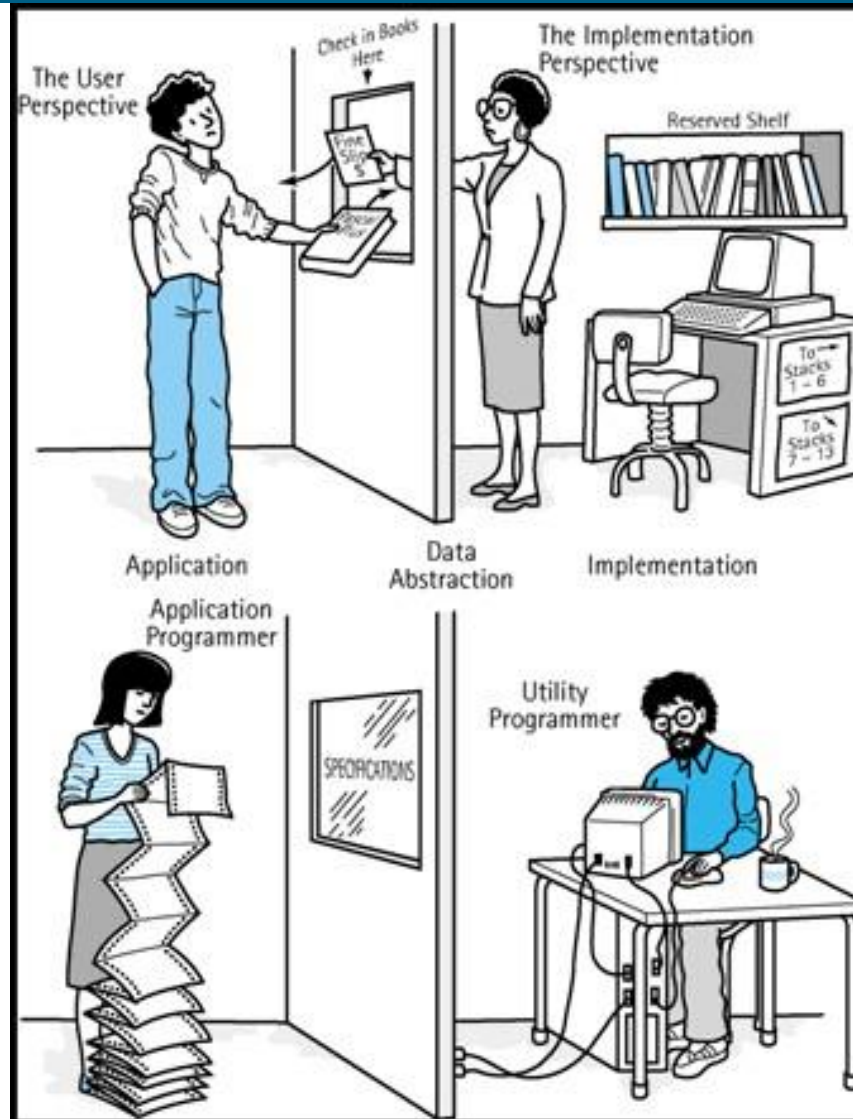
```
def dienTich(obj):
    return obj.dienTich()
```

```
v = Vương(3)
cn = ChuNhat(3, 4)
```

```
print(chuVi(v))
print(dienTich(cn))
```

# Tính trừu tượng

**Ý tưởng?**



*Hình (Internet)*

## ***Tính trừu tượng?***

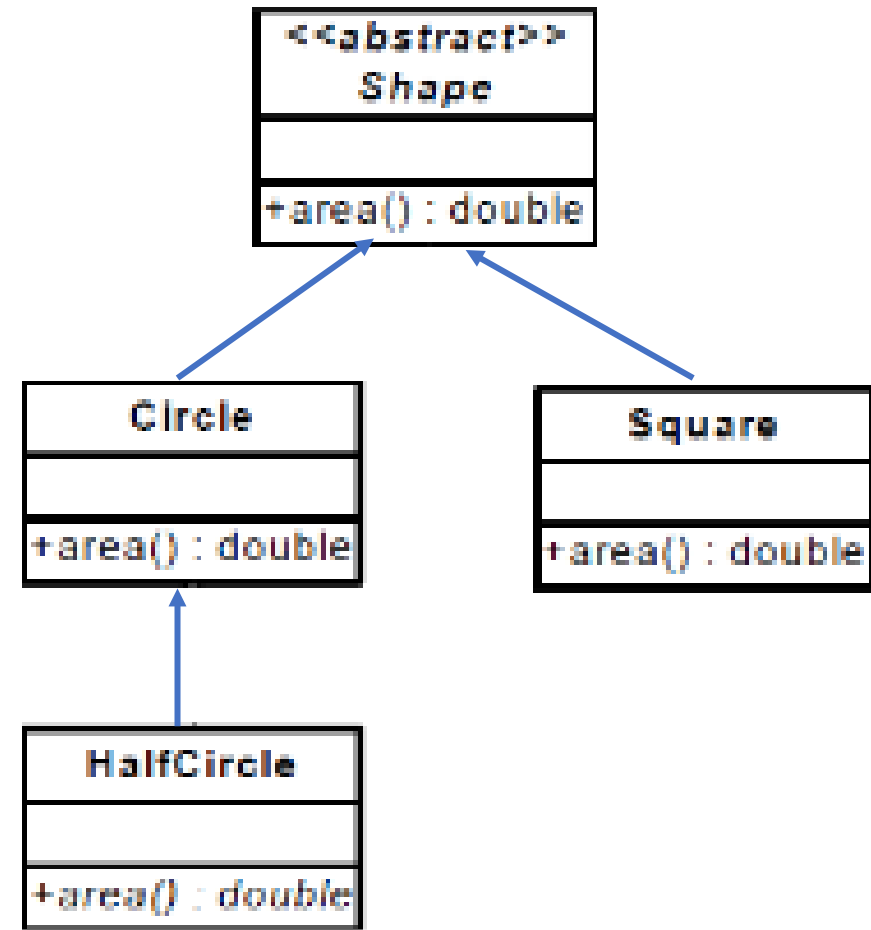
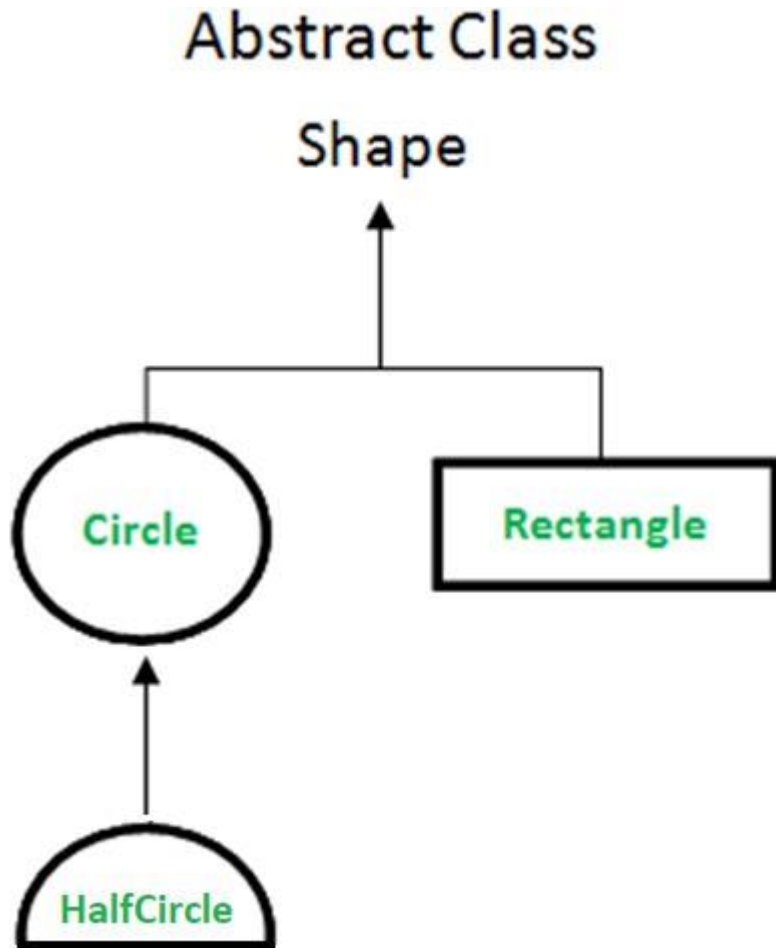
- Là tính chất không thể hiện cụ thể mà chỉ nêu tên vấn đề.
- Cho phép phớt lờ, không chú ý đến các đặc điểm của đối tượng. Thay vào đó, chỉ tập trung vào các đặc điểm cốt lõi của đối tượng mà bài toán đề cập.

## ***Ưu điểm***

- Tập trung vào những thông tin cốt lõi của đối tượng, thay vì quan tâm đến cách nó thực hiện
- Tính trừu tượng cung cấp nhiều chức năng mở rộng thêm nếu kết hợp với **tính đa hình và kế thừa**.

# Tính trừu tượng

## Phương pháp



# Tính trừu tượng

## Phương pháp

- Sử dụng module **abc** để:
  1. Kế thừa lớp trừu tượng **ABC**
    - Abstract Base Classes
  2. Decorator phương thức **abstractmethod**, hay khai báo ra các phương thức trừu tượng.
- Abstract class không phải là một class cụ thể, không thể khởi tạo, là một bản thiết kế cho các class khác.
- Đóng vai trò như một cái sườn để tạo ra các lớp con kế thừa từ cái sườn này.

```
from abc import ABC, abstractmethod
```

```
class TruuTuong(ABC):  
    @abstractmethod  
    def methodA(self):  
        pass  
    @abstractmethod  
    def methodB(self):  
        pass
```

*# Tạo các lớp kế thừa từ lớp trừu tượng để triển khai*

```
class TrienKhai(TruuTuong):  
    def methodA(self):  
        print('Triển khai chi tiết A')  
  
    def methodB(self):  
        print('Triển khai chi tiết B')
```

# Tính trừu tượng

## **Phương thức trừu tượng (Abstract method)**

- Sử dụng từ khóa `@abstractmethod` khi khai báo tên phương thức.
- Chỉ cần khai báo phương thức, không cần triển khai phương thức.

### **Tham khảo**

```
@abstractmethod
def methodA(self):
    pass
```

```
@abstractmethod
def methodB(self):
    pass
```

### **Chú ý:**

- Quyền hạn không để `private`
- Class kế thừa triển khai `override` phương thức trừu tượng này.

# Tính trừu tượng

## Ví dụ

```
from abc import ABC, abstractmethod

class Hình(ABC):
    @abstractmethod
    def chuVi(self):
        pass
    @abstractmethod
    def dienTich(self):
        pass
```

```
class Tron(Hình):
    def __init__(self, R):
        self.R = R

    def chuVi(self):
        return 2 * self.R * 3.14

    def dienTich(self):
        return self.R ** 2 * 3.14
```

```
class Vuong(Hình):
    def __init__(self, canh):
        self.canh = canh

    def chuVi(self):
        return self.canh * 4

    def dienTich(self):
        return self.canh ** 2
```

## Ví dụ

```
v = Vuong(6)
t = Tron(8)

print('Chu vi hình vuông   :', v.chuVi())
print('Diện tích hình vuông:', v.dienTich())

print('Chu vi đường tròn   :', t.chuVi())
print('Diện tích đường tròn:', t.dienTich())
```

## ***Bài tập***

- Tính chu vi và diện tích của hình vuông, chữ nhật... từ các điểm tọa độ.

## Bài tập QLVN: Áp dụng OOP (Tính đóng gói, kế thừa, đa hình, **trừu tượng**)

Viết chương trình quản lý nhân viên cho một công ty có tối đa 300 nhân viên. Công ty gồm 2 loại nhân viên: nhân viên văn phòng và nhân viên bán hàng. Công ty cần quản lý các thông tin sau của nhân viên: mã nhân viên, họ và tên, lương cơ bản, lương hằng tháng.

Lương hằng tháng tính bằng công thức sau:

- NV văn phòng = lương cơ bản + **số ngày là việc** \* 150\_000đ
- NV bán hàng = lương cơ bản + **số sản phẩm** \* 18\_000đ

Chương trình đáp ứng các yêu cầu sau:

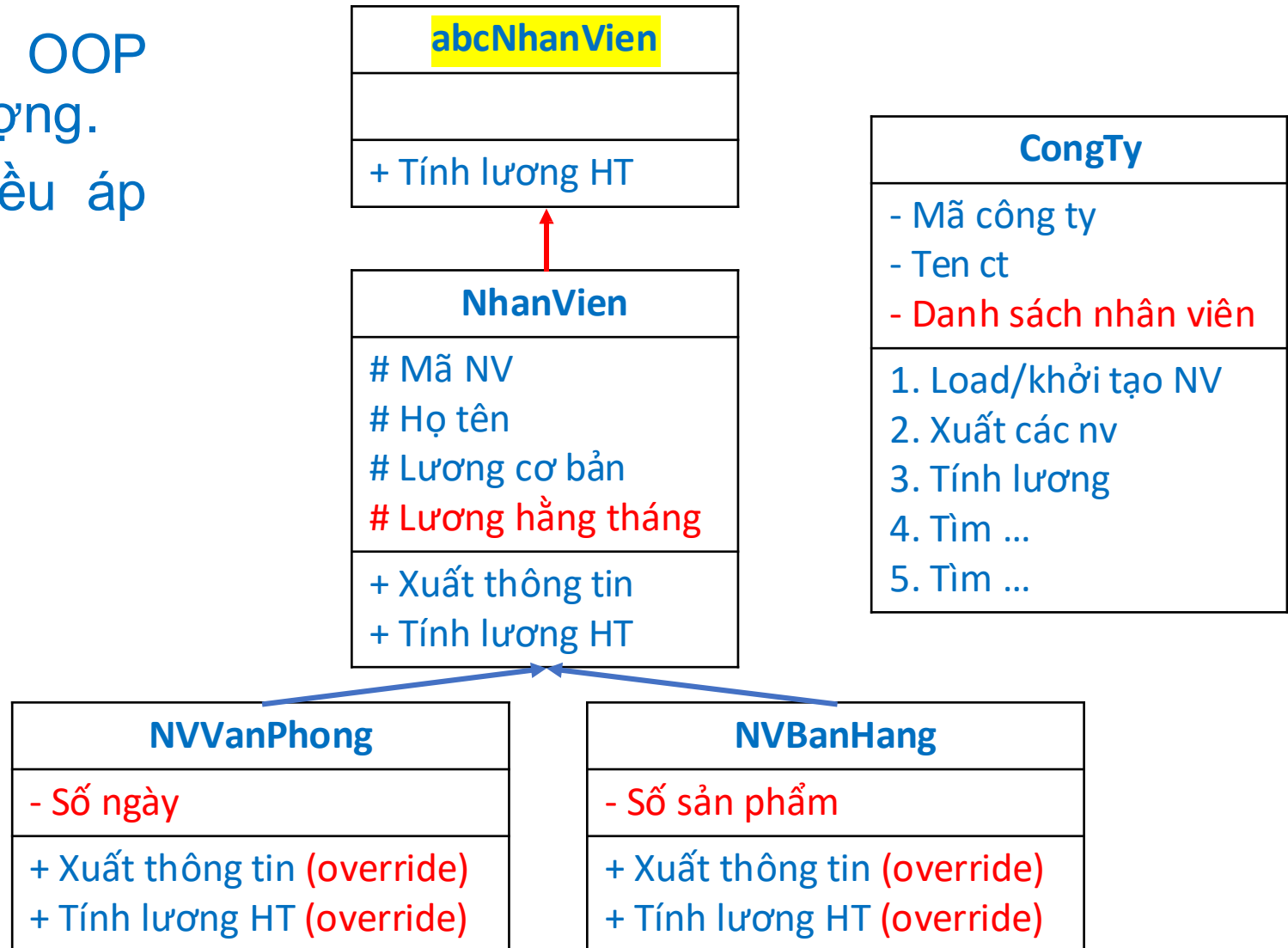
1. Khởi tạo tự động các nhân viên. Chỉ cung cấp: maNV, hoTen, luongCB.
2. Xuất các nhân viên (mã nhân viên, họ tên, lương cơ bản, **lương hằng tháng**).
3. Tính lương hằng tháng các nhân viên.
4. Tìm nhân viên theo mã nhân viên.
5. Cập nhật lương cơ bản theo mã nhân viên.
6. Cập nhật số ngày làm của nhân viên văn phòng theo mã nhân viên.
7. Tìm nhân viên có lương hằng tháng cao nhất.
8. Tìm nhân viên bán hàng có lương hằng tháng thấp nhất.

# Hướng dẫn

- Phạm vi kiến thức: Dùng OOP áp dụng thêm tính trừu tượng.
- Thiết kế: Mọi yêu cầu đều áp dụng OOP.
- Bao nhiêu class? => 5
- Liệt kê tên class? =>
  - **abcNhanVien**
  - NhanVien
  - NVVanPhong
  - NVBanHang
  - CongTy

Chú ý khi vẽ sơ đồ class

- private
- # protected
- + public





Thank you

# Hỏi - Đáp

